

EVIDENCIA DE DESEMPEÑO

Resolución a problemas algorítmicos
aplicando estructuras de almacenamiento

GA3-220501093-AA3-EV02

CAMPO	INFORMACIÓN
Programa	Análisis y Desarrollo de Software
Institución	Servicio Nacional de Aprendizaje SENA
Fase	Planeación
Naturaleza	Documento académico público

VERSIÓN PÚBLICA SIN DATOS PERSONALES

00 · PRESENTACIÓN

Alcance, método y evidencias

Cuatro soluciones ejecutables en JavaScript, acompañadas de análisis y pruebas.

RESULTADO

Los cuatro indicadores del instrumento se atienden mediante programas independientes, validaciones explícitas, funciones reutilizables y trece pruebas automatizadas.

Resolución a problemas algorítmicos aplicando estructuras de almacenamiento es el producto consolidado de esta evidencia de desempeño.

Ruta del documento

PROBLEMA	PROPÓSITO	ARCHIVO FUENTE
1	Perímetros y áreas de figuras planas	01_figuras_planas.js
2	Análisis de diez edades	02_analisis_edades.js
3	Mezcla ordenada de dos vectores	03_mezclar_vectores.js
4	Encuesta musical persistente	04_encuesta_musical.js

Criterios transversales

- Cada entrada se valida antes de participar en un cálculo o almacenarse.
- La lógica de negocio se separa de la interfaz interactiva para facilitar pruebas.
- Los arreglos se procesan sin mutaciones accidentales y con complejidad documentada.
- La encuesta musical incorpora persistencia JSON y operaciones de ingreso, consulta, modificación y eliminación.

Los listados incluidos a continuación corresponden a las fuentes verificadas por Node.js. Los ejemplos de prueba emplean datos sintéticos y no contienen información del aprendiz.

01 · PROBLEMA 1

Perímetros y áreas de figuras planas

Calcular perímetro y área de triángulo, rectángulo, cuadrado y círculo.

Estrategia de solución

Se usa un despachador por tipo de figura y una función independiente para cada fórmula. La circunferencia se calcula correctamente como $2 \times \pi \times \text{radio}$.

EJEMPLO VERIFICABLE

Con radio 3, el círculo produce un perímetro aproximado de 18,8496 y un área de 28,2743.

Controles de entrada y consistencia

- Todas las medidas deben ser numéricas, finitas y mayores que cero.
- Los tres lados del triángulo deben cumplir la desigualdad triangular.
- Solo se aceptan las cuatro figuras contempladas por la guía.

COMPROBACIÓN AUTOMÁTICA

Tres pruebas cubren cálculos, despacho, formato y dominios inválidos.

Código fuente completo · 01_figuras_planas.js

```
"use strict";

const readline = require("node:readline/promises");
const { stdin: input, stdout: output } = require("node:process");

const TIPOS_FIGURA = Object.freeze({
  TRIANGULO: "triángulo",
  RECTANGULO: "rectángulo",
  CUADRADO: "cuadrado",
  CIRCULO: "círculo",
});

function exigirPositivo(valor, nombre) {
  if (typeof valor !== "number" || !Number.isFinite(valor) || valor <= 0) {
    throw new RangeError(`${nombre} debe ser un número positivo.`);
  }
  return valor;
}

function validarTriangulo(ladoA, ladoB, ladoC) {
  exigirPositivo(ladoA, "El lado A");
  exigirPositivo(ladoB, "El lado B");
  exigirPositivo(ladoC, "El lado C");

  const esValido =
    ladoA + ladoB > ladoC &&
    ladoA + ladoC > ladoB &&
    ladoB + ladoC > ladoA;

  if (!esValido) {
    throw new RangeError(
      "Los lados no forman un triángulo: la suma de dos lados debe superar al tercero."
    );
  }
  return true;
}

function calcularTriangulo({ ladoA, ladoB, ladoC, altura }) {
  validarTriangulo(ladoA, ladoB, ladoC);
  exigirPositivo(altura, "La altura");
  return Object.freeze({
    figura: TIPOS_FIGURA.TRIANGULO,
    perimetro: ladoA + ladoB + ladoC,
    area: (ladoB * altura) / 2,
  });
}

function calcularRectangulo({ base, altura }) {
  exigirPositivo(base, "La base");
  exigirPositivo(altura, "La altura");
  return Object.freeze({
    figura: TIPOS_FIGURA.RECTANGULO,
    perimetro: 2 * (base + altura),
    area: base * altura,
  });
}

function calcularCuadrado({ lado }) {
```

```

    exigirPositivo(lado, "El lado");
    return Object.freeze({
      figura: TIPOS_FIGURA.CUADRADO,
      perimetro: 4 * lado,
      area: lado ** 2,
    });
  }

function calcularCirculo({ radio }) {
  exigirPositivo(radio, "El radio");
  return Object.freeze({
    figura: TIPOS_FIGURA.CIRCULO,
    perimetro: 2 * Math.PI * radio,
    area: Math.PI * radio ** 2,
  });
}

function normalizarTipoFigura(tipo) {
  const valor = String(tipo).trim().toLowerCase();
  const equivalencias = {
    "1": TIPOS_FIGURA.TRIANGULO,
    triangulo: TIPOS_FIGURA.TRIANGULO,
    "2": TIPOS_FIGURA.RECTANGULO,
    rectangulo: TIPOS_FIGURA.RECTANGULO,
    "3": TIPOS_FIGURA.CUADRADO,
    cuadrado: TIPOS_FIGURA.CUADRADO,
    "4": TIPOS_FIGURA.CIRCULO,
    circulo: TIPOS_FIGURA.CIRCULO,
  };
  const normalizado = equivalencias[valor];
  if (!normalizado) {
    throw new RangeError("La figura debe ser triángulo, rectángulo, cuadrado o círculo.");
  }
  return normalizado;
}

function calcularFigura(tipo, medidas) {
  const figura = normalizarTipoFigura(tipo);
  const calculadoras = {
    [TIPOS_FIGURA.TRIANGULO]: calcularTriangulo,
    [TIPOS_FIGURA.RECTANGULO]: calcularRectangulo,
    [TIPOS_FIGURA.CUADRADO]: calcularCuadrado,
    [TIPOS_FIGURA.CIRCULO]: calcularCirculo,
  };
  return calculadoras[figura](medidas);
}

async function leerNumeroPositivo(interfaz, mensaje) {
  while (true) {
    const respuesta = (await interfaz.question(mensaje)).trim().replace(",", ".");
    const valor = Number(respuesta);
    if (Number.isFinite(valor) && valor > 0) {
      return valor;
    }
    console.log("Entrada inválida. Escriba un número mayor que cero.");
  }
}

async function leerFigura(interfaz) {
  while (true) {
    console.log("\n1. Triángulo\n2. Rectángulo\n3. Cuadrado\n4. Círculo");
    const opcion = await interfaz.question("Seleccione una figura: ");
    try {
      return normalizarTipoFigura(opcion);
    } catch (error) {
      console.log(error.message);
    }
  }
}

async function solicitarMedidas(interfaz, figura) {
  if (figura === TIPOS_FIGURA.TRIANGULO) {
    while (true) {
      const ladoA = await leerNumeroPositivo(interfaz, "Lado A: ");
      const ladoB = await leerNumeroPositivo(interfaz, "Lado B (base): ");
      const ladoC = await leerNumeroPositivo(interfaz, "Lado C: ");
      const altura = await leerNumeroPositivo(interfaz, "Altura respecto al lado B: ");
      try {
        validarTriangulo(ladoA, ladoB, ladoC);
        return { ladoA, ladoB, ladoC, altura };
      } catch (error) {
        console.log(`${error.message} Ingrese nuevamente las medidas.`);
      }
    }
  }
  if (figura === TIPOS_FIGURA.RECTANGULO) {
    return {
      base: await leerNumeroPositivo(interfaz, "Base: "),
      altura: await leerNumeroPositivo(interfaz, "Altura: "),
    };
  }
}

```

```
if (figura === TIPOS_FIGURA.CUADRADO) {
  return { lado: await leerNumeroPositivo(interfaz, "Lado: ") };
}
return { radio: await leerNumeroPositivo(interfaz, "Radio: ") };
}

function formatearResultado(resultado) {
  return [
    `Figura: ${resultado.figura}`,
    `Perímetro: ${resultado.perimetro.toFixed(2)}`,
    `Área: ${resultado.area.toFixed(2)}`,
  ].join("\n");
}

async function main() {
  const interfaz = readline.createInterface({ input, output });
  try {
    console.log("Cálculo de área y perímetro de figuras planas");
    const figura = await leerFigura(interfaz);
    const medidas = await solicitarMedidas(interfaz, figura);
    console.log(`\n${formatearResultado(calcularFigura(figura, medidas))}`);
  } finally {
    interfaz.close();
  }
}

module.exports = {
  TIPOS_FIGURA,
  exigirPositivo,
  validarTriangulo,
  calcularTriangulo,
  calcularRectangulo,
  calcularCuadrado,
  calcularCirculo,
  normalizarTipoFigura,
  calcularFigura,
  formatearResultado,
};

if (require.main === module) {
  main().catch((error) => {
    console.error(`Error: ${error.message}`);
    process.exitCode = 1;
  });
}
```

02 · PROBLEMA 2

Análisis de diez edades

Leer diez edades y obtener menores, adultos, adultos mayores, mínimo, máximo y promedio.

Estrategia de solución

El arreglo se valida antes de calcular las estadísticas en una sola reducción. Las categorías son excluyentes: 1–17, 18–59 y 60 años o más.

EJEMPLO VERIFICABLE

Para [10, 17, 18, 25, 40, 59, 60, 70, 80, 90], el mínimo es 10, el máximo 90 y el promedio 46,9.

Controles de entrada y consistencia

- El arreglo debe contener exactamente diez elementos.
- Cada edad debe ser un entero entre 1 y 120.
- El promedio conserva precisión y se presenta con formato legible.

COMPROBACIÓN AUTOMÁTICA

Dos pruebas verifican estadísticas, límites y rechazo de edades inválidas.

Código fuente completo · 02_analisis_edades.js

```
"use strict";

const readline = require("node:readline/promises");
const { stdin: input, stdout: output } = require("node:process");

const CANTIDAD_EDADES = 10;
const EDAD_MINIMA = 1;
const EDAD_MAXIMA = 120;

function validarEdad(edad) {
  if (!Number.isInteger(edad) || edad < EDAD_MINIMA || edad > EDAD_MAXIMA) {
    throw new RangeError(
      `La edad debe ser un entero entre ${EDAD_MINIMA} y ${EDAD_MAXIMA}.`,
    );
  }
  return edad;
}

function validarEdades(edades) {
  if (!Array.isArray(edades) || edades.length !== CANTIDAD_EDADES) {
    throw new RangeError(`Se requieren exactamente ${CANTIDAD_EDADES} edades.`);
  }
  return edades.map(validarEdad);
}

function analizarEdades(edades) {
  const valores = validarEdades(edades);
  const menoresDe18 = valores.filter((edad) => edad < 18);
  const entre18Y59 = valores.filter((edad) => edad >= 18 && edad <= 59);
  const mayoresOigualesA60 = valores.filter((edad) => edad >= 60);
  const suma = valores.reduce((acumulado, edad) => acumulado + edad, 0);

  return Object.freeze({
    edades: Object.freeze([...valores]),
    grupos: Object.freeze({
      menoresDe18: Object.freeze(menoresDe18),
      entre18Y59: Object.freeze(entre18Y59),
      mayoresOigualesA60: Object.freeze(mayoresOigualesA60),
    }),
    conteos: Object.freeze({
      menoresDe18: menoresDe18.length,
      entre18Y59: entre18Y59.length,
      mayoresOigualesA60: mayoresOigualesA60.length,
    }),
    minimo: Math.min(...valores),
    maximo: Math.max(...valores),
    promedio: suma / valores.length,
  });
}

async function leerEdad(interfaz, posicion) {
  while (true) {
    const respuesta = (await interfaz.question(`Edad ${posicion}: `)).trim();
    const edad = Number(respuesta);
    try {
      return validarEdad(edad);
    } catch (error) {
      console.log(`${error.message} Vuelva a intentarlo.`);
    }
  }
}
```

```
    }  
  }  
}  
  
function mostrarGrupo(etiqueta, edades) {  
  const contenido = edades.length > 0 ? edades.join(", ") : "ninguna";  
  console.log(`${etiqueta}: ${edades.length} (${contenido})`);  
}  
  
async function main() {  
  const interfaz = readline.createInterface({ input, output });  
  try {  
    console.log(`Análisis de ${CANTIDAD_EDADES} edades`);  
    const edades = [];  
    for (let posicion = 1; posicion <= CANTIDAD_EDADES; posicion += 1) {  
      edades.push(await leerEdad(interfaz, posicion));  
    }  
  
    const resultado = analizarEdades(edades);  
    console.log("\nDistribución por grupos");  
    mostrarGrupo("Menores de 18", resultado.grupos.menoresDe18);  
    mostrarGrupo("Entre 18 y 59", resultado.grupos.entre18Y59);  
    mostrarGrupo("Mayores o iguales a 60", resultado.grupos.mayoresOigualesA60);  
    console.log(`Edad mínima: ${resultado.minimo}`);  
    console.log(`Edad máxima: ${resultado.maximo}`);  
    console.log(`Promedio: ${resultado.promedio.toFixed(2)}`);  
  } finally {  
    interfaz.close();  
  }  
}  
  
module.exports = {  
  CANTIDAD_EDADES,  
  EDAD_MINIMA,  
  EDAD_MAXIMA,  
  validarEdad,  
  validarEdades,  
  analizarEdades,  
};  
  
if (require.main === module) {  
  main().catch((error) => {  
    console.error(`Error: ${error.message}`);  
    process.exitCode = 1;  
  });  
}
```

03 · PROBLEMA 3

Mezcla ordenada de dos vectores

Combinar dos vectores ascendentes, de máximo cinco elementos cada uno, sin perder el orden.

Estrategia de solución

Se aplica el algoritmo de mezcla con dos índices. Su costo es lineal $O(n + m)$, conserva duplicados y evita ordenar de nuevo el resultado.

EJEMPLO VERIFICABLE

La mezcla de [1, 4, 7] y [2, 4, 9] genera [1, 2, 4, 4, 7, 9].

Controles de entrada y consistencia

- Cada vector debe tener entre uno y cinco enteros.
- Los valores de entrada deben encontrarse en orden ascendente.
- La función devuelve un arreglo nuevo y no modifica las entradas.

COMPROBACIÓN AUTOMÁTICA

Dos pruebas cubren mezcla, duplicados, tamaños, tipos y orden inválido.

Código fuente completo · 03_mezclar_vectores.js

```
"use strict";

const readline = require("node:readline/promises");
const { stdin: input, stdout: output } = require("node:process");

const LONGITUD_MINIMA = 1;
const LONGITUD_MAXIMA = 5;

function validarLongitud(longitud) {
  if (
    !Number.isInteger(longitud) ||
    longitud < LONGITUD_MINIMA ||
    longitud > LONGITUD_MAXIMA
  ) {
    throw new RangeError(
      `La longitud debe ser un entero entre ${LONGITUD_MINIMA} y ${LONGITUD_MAXIMA}.`,
    );
  }
  return longitud;
}

function validarVectorAscendente(vector, nombre = "El vector") {
  if (!Array.isArray(vector)) {
    throw new TypeError(`${nombre} debe ser un arreglo.`);
  }
  validarLongitud(vector.length);

  for (let indice = 0; indice < vector.length; indice += 1) {
    if (!Number.isInteger(vector[indice])) {
      throw new TypeError(`${nombre} solo puede contener números enteros.`);
    }
    if (indice > 0 && vector[indice] < vector[indice - 1]) {
      throw new RangeError(`${nombre} debe estar ordenado ascendentemente.`);
    }
  }
  return [...vector];
}

function mezclarVectoresAscendentes(vectorA, vectorB) {
  const primero = validarVectorAscendente(vectorA, "El vector A");
  const segundo = validarVectorAscendente(vectorB, "El vector B");
  const mezcla = [];
  let indiceA = 0;
  let indiceB = 0;

  while (indiceA < primero.length && indiceB < segundo.length) {
    if (primero[indiceA] <= segundo[indiceB]) {
      mezcla.push(primero[indiceA]);
      indiceA += 1;
    } else {
      mezcla.push(segundo[indiceB]);
      indiceB += 1;
    }
  }

  while (indiceA < primero.length) {
    mezcla.push(primero[indiceA]);
    indiceA += 1;
  }
}
```



```

    }
    while (indiceB < segundo.length) {
        mezcla.push(segundo[indiceB]);
        indiceB += 1;
    }
    return mezcla;
}

async function leerEntero(interfaz, mensaje) {
    while (true) {
        const respuesta = (await interfaz.question(mensaje)).trim();
        const valor = Number(respuesta);
        if (Number.isInteger(valor)) {
            return valor;
        }
        console.log("Entrada inválida. Escriba un número entero.");
    }
}

async function leerLongitud(interfaz, nombre) {
    while (true) {
        const longitud = await leerEntero(
            interfaz,
            `Cantidad de elementos del vector ${nombre} (${LONGITUD_MINIMA}-${LONGITUD_MAXIMA}): `,
        );
        try {
            return validarLongitud(longitud);
        } catch (error) {
            console.log(error.message);
        }
    }
}

async function leerVectorAscendente(interfaz, nombre) {
    const longitud = await leerLongitud(interfaz, nombre);
    const vector = [];
    console.log(`Ingrese el vector ${nombre} en orden ascendente; se permiten duplicados.`);

    for (let posicion = 0; posicion < longitud; posicion += 1) {
        while (true) {
            const minimo = posicion === 0 ? "sin límite inferior" : `mínimo ${vector[posicion - 1]}`;
            const valor = await leerEntero(
                interfaz,
                `Elemento ${posicion + 1} (${minimo}): `,
            );
            if (posicion === 0 || valor >= vector[posicion - 1]) {
                vector.push(valor);
                break;
            }
            console.log("El valor rompe el orden ascendente. Ingrésele nuevamente.");
        }
    }
    return vector;
}

async function main() {
    const interfaz = readline.createInterface({ input, output });
    try {
        console.log("Mezcla lineal de dos vectores ascendentes");
        const vectorA = await leerVectorAscendente(interfaz, "A");
        const vectorB = await leerVectorAscendente(interfaz, "B");
        const mezcla = mezclarVectoresAscendentes(vectorA, vectorB);
        console.log(`\nVector A: [${vectorA.join(", ")}]`);
        console.log(`Vector B: [${vectorB.join(", ")}]`);
        console.log(`Mezcla: [${mezcla.join(", ")}]`);
    } finally {
        interfaz.close();
    }
}

module.exports = {
    LONGITUD_MINIMA,
    LONGITUD_MAXIMA,
    validarLongitud,
    validarVectorAscendente,
    mezclarVectoresAscendentes,
};

if (require.main === module) {
    main().catch((error) => {
        console.error(`Error: ${error.message}`);
        process.exitCode = 1;
    });
}

```

04 · PROBLEMA 4

Encuesta musical persistente

Administrar los datos de hasta seis personas y sus canciones favoritas mediante un archivo JSON.

Estrategia de solución

La lógica de dominio permanece separada del menú interactivo y de la persistencia. Se implementan alta, consulta, listado, modificación, eliminación, guardado y recarga.

EJEMPLO VERIFICABLE

Un registro sintético DEMO-001 puede agregarse, consultarse, modificarse y eliminarse sin afectar los demás datos.

Controles de entrada y consistencia

- La identificación es única y cada campo se normaliza antes de almacenarse.
- Cada persona registra entre una y tres canciones con artista y título.
- El reemplazo del archivo se realiza de forma atómica y se rechaza JSON dañado.

COMPROBACIÓN AUTOMÁTICA

Seis pruebas cubren validación, CRUD, límite de registros y persistencia segura.

Código fuente completo · 04_encuesta_musical.js

```
"use strict";

const fs = require("node:fs/promises");
const path = require("node:path");
const readline = require("node:readline/promises");
const { stdin: input, stdout: output } = require("node:process");

const MAX_PERSONAS = 6;
const MIN_CANCIONES = 1;
const MAX_CANCIONES = 3;
const ARCHIVO_PREDETERMINADO = path.join(__dirname, "encuesta_musical.local.json");

function exigirTexto(valor, campo, maximo = 100) {
  if (typeof valor !== "string") {
    throw new TypeError(`${campo} debe ser texto.`);
  }
  const limpio = valor.trim();
  if (limpio.length === 0 || limpio.length > maximo) {
    throw new RangeError(`${campo} debe contener entre 1 y ${maximo} caracteres.`);
  }
  return limpio;
}

function validarNumeroIdentificacion(valor) {
  const limpio = exigirTexto(valor, "El número de identificación", 30);
  if (!/^[A-Za-z0-9-]+$/ .test(limpio)) {
    throw new RangeError(
      "El número de identificación solo admite letras, números y guiones."
    );
  }
  return limpio;
}

function validarFecha(fecha) {
  const limpia = exigirTexto(fecha, "La fecha de nacimiento", 10);
  const coincidencia = /^(\d{4})-(\d{2})-(\d{2})$/ .exec(limpia);
  if (!coincidencia) {
    throw new RangeError("La fecha de nacimiento debe usar el formato AAAA-MM-DD.");
  }
  const [, anioTexto, mesTexto, diaTexto] = coincidencia;
  const anio = Number(anioTexto);
  const mes = Number(mesTexto);
  const dia = Number(diaTexto);
  const fechaUtc = new Date(Date.UTC(anio, mes - 1, dia));
  const esMismaFecha =
    fechaUtc.getUTCFullYear() === anio &&
    fechaUtc.getUTCMonth() === mes - 1 &&
    fechaUtc.getUTCDate() === dia;
  if (!esMismaFecha) {
    throw new RangeError("La fecha de nacimiento no existe en el calendario.");
  }
  if (fechaUtc > new Date()) {
    throw new RangeError("La fecha de nacimiento no puede estar en el futuro.");
  }
  return limpia;
}

function validarCorreo(correoElectronico) {
```

```
const limpio = exigirTexto(correoElectronico, "El correo electrónico", 120);
const patron = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!patron.test(limpio)) {
  throw new RangeError("El correo electrónico no tiene un formato válido.");
}
return limpio;
}

function normalizarCancion(cancion) {
  if (!cancion || typeof cancion !== "object" || Array.isArray(cancion)) {
    throw new TypeError("Cada canción debe contener artista y título.");
  }
  return Object.freeze({
    artista: exigirTexto(cancion.artista, "El artista", 100),
    titulo: exigirTexto(cancion.titulo, "El título", 120),
  });
}

function validarCanciones(cancionesFavoritas) {
  if (
    !Array.isArray(cancionesFavoritas) ||
    cancionesFavoritas.length < MIN_CANCIONES ||
    cancionesFavoritas.length > MAX_CANCIONES
  ) {
    throw new RangeError(
      `Se requieren entre ${MIN_CANCIONES} y ${MAX_CANCIONES} canciones favoritas.`
    );
  }
  return Object.freeze(cancionesFavoritas.map(normalizarCancion));
}

function normalizarPersona(persona) {
  if (!persona || typeof persona !== "object" || Array.isArray(persona)) {
    throw new TypeError("La persona debe representarse mediante un objeto.");
  }
  return Object.freeze({
    nombreCompleto: exigirTexto(persona.nombreCompleto, "El nombre completo", 100),
    numeroIdentificacion: validarNumeroIdentificacion(persona.numeroIdentificacion),
    fechaNacimiento: validarFecha(persona.fechaNacimiento),
    correoElectronico: validarCorreo(persona.correoElectronico),
    ciudadResidencia: exigirTexto(persona.ciudadResidencia, "La ciudad de residencia", 80),
    ciudadOrigen: exigirTexto(persona.ciudadOrigen, "La ciudad de origen", 80),
    cancionesFavoritas: validarCanciones(persona.cancionesFavoritas),
  });
}

function validarEncuesta(personas) {
  if (!Array.isArray(personas)) {
    throw new TypeError("La encuesta debe ser un arreglo de personas.");
  }
  if (personas.length > MAX_PERSONAS) {
    throw new RangeError(`La encuesta admite como máximo ${MAX_PERSONAS} personas.`);
  }
  const normalizadas = personas.map(normalizarPersona);
  const identificaciones = new Set();
  for (const persona of normalizadas) {
    const clave = persona.numeroIdentificacion.toLowerCase();
    if (identificaciones.has(clave)) {
      throw new RangeError("Cada número de identificación debe ser único.");
    }
    identificaciones.add(clave);
  }
  return normalizadas;
}

function agregarPersona(personas, nuevaPersona) {
  const actuales = validarEncuesta(personas);
  if (actuales.length >= MAX_PERSONAS) {
    throw new RangeError(`No se pueden registrar más de ${MAX_PERSONAS} personas.`);
  }
  return validarEncuesta([...actuales, normalizarPersona(nuevaPersona)]);
}

function indiceDesdePosicion(personas, posicion) {
  if (!Number.isInteger(posicion) || posicion < 1 || posicion > personas.length) {
    throw new RangeError(`La posición debe estar entre 1 y ${personas.length}.`);
  }
  return posicion - 1;
}

function obtenerPersonaPorPosicion(personas, posicion) {
  const actuales = validarEncuesta(personas);
  const indice = indiceDesdePosicion(actuales, posicion);
  return actuales[indice];
}

function modificarPersona(personas, posicion, personaActualizada) {
  const actuales = validarEncuesta(personas);
  const indice = indiceDesdePosicion(actuales, posicion);
  const copia = [...actuales];
  copia[indice] = normalizarPersona(personaActualizada);
}
```

```

    return validarEncuesta(copia);
}

function eliminarPersona(personas, posicion) {
    const actuales = validarEncuesta(personas);
    const indice = indiceDesdePosicion(actuales, posicion);
    return actuales.filter( (_, indiceActual) => indiceActual !== indice);
}

function listarPersonas(personas) {
    return validarEncuesta(personas).map((persona, indice) =>
        Object.freeze({
            posicion: indice + 1,
            nombreCompleto: persona.nombreCompleto,
            numeroIdentificacion: persona.numeroIdentificacion,
            cantidadCanciones: persona.cancionesFavoritas.length,
        })),
    );
}

async function cargarEncuesta(rutaArchivo = ARCHIVO_PREDETERMINADO) {
    try {
        const contenido = await fs.readFile(rutaArchivo, "utf8");
        return validarEncuesta(JSON.parse(contenido));
    } catch (error) {
        if (error.code === "ENOENT") {
            return [];
        }
        if (error instanceof SyntaxError) {
            throw new Error("El archivo JSON no contiene información válida.");
        }
        throw error;
    }
}

async function guardarEncuesta(personas, rutaArchivo = ARCHIVO_PREDETERMINADO) {
    const normalizadas = validarEncuesta(personas);
    const temporal = `${rutaArchivo}.tmp`;
    const contenido = ` ${JSON.stringify(normalizadas, null, 2)}\n`;
    try {
        await fs.writeFile(temporal, contenido, { encoding: "utf8", mode: 0o600 });
        await fs.rename(temporal, rutaArchivo);
        await fs.chmod(rutaArchivo, 0o600);
    } finally {
        await fs.rm(temporal, { force: true });
    }
}

async function leerTexto(interfaz, mensaje, validador = null) {
    while (true) {
        const respuesta = await interfaz.question(mensaje);
        try {
            return validador ? validador(respuesta) : exigirTexto(respuesta, "El valor");
        } catch (error) {
            console.log(` ${error.message} Vuelva a intentarlo.`);
        }
    }
}

async function leerEnteroEnRango(interfaz, mensaje, minimo, maximo) {
    while (true) {
        const respuesta = (await interfaz.question(mensaje)).trim();
        const valor = Number(respuesta);
        if (Number.isInteger(valor) && valor >= minimo && valor <= maximo) {
            return valor;
        }
        console.log(`Escriba un entero entre ${minimo} y ${maximo}.`);
    }
}

async function solicitarCanciones(interfaz) {
    const cantidad = await leerEnteroEnRango(
        interfaz,
        `Cantidad de canciones favoritas (${MIN_CANCIONES}-${MAX_CANCIONES}): `,
        MIN_CANCIONES,
        MAX_CANCIONES,
    );
    const canciones = [];
    for (let posicion = 1; posicion <= cantidad; posicion += 1) {
        console.log(`Canción ${posicion}`);
        canciones.push({
            artista: await leerTexto(interfaz, " Artista: "),
            titulo: await leerTexto(interfaz, " Título: "),
        });
    }
    return canciones;
}

async function solicitarPersona(interfaz) {
    return normalizarPersona({
        nombreCompleto: await leerTexto(interfaz, "Nombre completo: "),
    });
}

```

```

    numeroIdentificacion: await leerTexto(
      interfaz,
      "Número de identificación: ",
      validarNumeroIdentificacion,
    ),
    fechaNacimiento: await leerTexto(
      interfaz,
      "Fecha de nacimiento (AAAA-MM-DD): ",
      validarFecha,
    ),
    correoElectronico: await leerTexto(
      interfaz,
      "Correo electrónico: ",
      validarCorreo,
    ),
    ciudadResidencia: await leerTexto(interfaz, "Ciudad de residencia: "),
    ciudadOrigen: await leerTexto(interfaz, "Ciudad de origen: "),
    cancionesFavoritas: await solicitarCanciones(interfaz),
  });
}

function imprimirPersona(persona, posicion) {
  console.log(`\nPersona ${posicion}`);
  console.log(`Nombre: ${persona.nombreCompleto}`);
  console.log(`Número de identificación: ${persona.numeroIdentificacion}`);
  console.log(`Fecha de nacimiento: ${persona.fechaNacimiento}`);
  console.log(`Correo electrónico: ${persona.correoElectronico}`);
  console.log(`Ciudad de residencia: ${persona.ciudadResidencia}`);
  console.log(`Ciudad de origen: ${persona.ciudadOrigen}`);
  console.log("Canciones favoritas:");
  persona.cancionesFavoritas.forEach((cancion, indice) => {
    console.log(`  ${indice + 1}. ${cancion.titulo} - ${cancion.artista}`);
  });
}

function imprimirListado(personas) {
  const listado = listarPersonas(personas);
  if (listado.length === 0) {
    console.log("No hay personas registradas.");
    return;
  }
  console.log("\nPersonas registradas");
  for (const persona of listado) {
    console.log(
      `${persona.posicion}. ${persona.nombreCompleto} | identificación: ${persona.numeroIdentificacion} |`
    );
    console.log(`canciones: ${persona.cantidadCanciones}`);
  }
}

async function leerPosicion(interfaz, personas, accion) {
  if (personas.length === 0) {
    console.log("No hay personas registradas.");
    return null;
  }
  imprimirListado(personas);
  return leerEnteroEnRango(
    interfaz,
    `Posición que desea ${accion} (1-${personas.length}): `,
    1,
    personas.length,
  );
}

async function ejecutarMenu(interfaz, rutaArchivo) {
  let personas = await cargarEncuesta(rutaArchivo);
  let continuar = true;

  while (continuar) {
    console.log(
      "\nEncuesta musical\n" +
      "1. Agregar persona\n" +
      "2. Mostrar persona por posición\n" +
      "3. Modificar persona\n" +
      "4. Eliminar persona\n" +
      "5. Listar personas\n" +
      "0. Salir",
    );
    const opcion = await leerEnteroEnRango(interfaz, "Opción: ", 0, 5);

    if (opcion === 1) {
      try {
        personas = agregarPersona(personas, await solicitarPersona(interfaz));
        await guardarEncuesta(personas, rutaArchivo);
        console.log("Persona agregada y encuesta guardada.");
      } catch (error) {
        console.log(`No se pudo agregar: ${error.message}`);
      }
    } else if (opcion === 2) {
      const posicion = await leerPosicion(interfaz, personas, "mostrar");
      if (posicion !== null) {

```

```
        imprimirPersona(obtenerPersonaPorPosicion(personas, posicion), posicion);
    }
} else if (opcion === 3) {
    const posicion = await leerPosicion(interfaz, personas, "modificar");
    if (posicion !== null) {
        try {
            personas = modificarPersona(
                personas,
                posicion,
                await solicitarPersona(interfaz),
            );
            await guardarEncuesta(personas, rutaArchivo);
            console.log("Persona modificada y encuesta guardada.");
        } catch (error) {
            console.log(`No se pudo modificar: ${error.message}`);
        }
    }
} else if (opcion === 4) {
    const posicion = await leerPosicion(interfaz, personas, "eliminar");
    if (posicion !== null) {
        personas = eliminarPersona(personas, posicion);
        await guardarEncuesta(personas, rutaArchivo);
        console.log("Persona eliminada y encuesta guardada.");
    }
} else if (opcion === 5) {
    imprimirListado(personas);
} else {
    continuar = false;
}
}
}

async function main() {
    const interfaz = readline.createInterface({ input, output });
    const rutaArchivo = process.env.ENCUESTA_ARCHIVO || ARCHIVO_PREDETERMINADO;
    try {
        await ejecutarMenu(interfaz, rutaArchivo);
    } finally {
        interfaz.close();
    }
}

module.exports = {
    MAX_PERSONAS,
    MIN_CANCIONES,
    MAX_CANCIONES,
    ARCHIVO_PREDETERMINADO,
    exigirTexto,
    validarNumeroIdentificacion,
    validarFecha,
    validarCorreo,
    normalizarCancion,
    validarCanciones,
    normalizarPersona,
    validarEncuesta,
    agregarPersona,
    obtenerPersonaPorPosicion,
    modificarPersona,
    eliminarPersona,
    listarPersonas,
    cargarEncuesta,
    guardarEncuesta,
};

if (require.main === module) {
    main().catch((error) => {
        console.error(`Error: ${error.message}`);
        process.exitCode = 1;
    });
}
```

05 · VERIFICACIÓN

Matriz final de cumplimiento

Trazabilidad directa entre la lista de chequeo, el código y las pruebas.

IND.	EVIDENCIA	RESULTADO
1	Problema 1: cuatro figuras, fórmulas y validaciones.	3 pruebas aprobadas
2	Problema 2: diez edades, categorías y estadísticas.	2 pruebas aprobadas
3	Problema 3: mezcla lineal de vectores ascendentes.	2 pruebas aprobadas
4	Problema 4: encuesta, CRUD y persistencia JSON.	6 pruebas aprobadas

EJECUCIÓN VERIFICADA

13 de 13 pruebas aprobadas, 0 fallidas. Además, Node.js comprobó la sintaxis de las cuatro fuentes.

Conclusiones

- Las funciones pequeñas y puras simplifican la lectura, el mantenimiento y las pruebas.
- Los arreglos permiten representar colecciones y resolver estadísticas o mezclas de forma eficiente.
- Separar dominio, interacción y persistencia reduce errores al administrar información.
- La validación previa evita resultados incoherentes y protege el archivo de datos.

Referencias

[1] SENA. Guía de aprendizaje GA3-220501093-AA3, páginas 9–10.

[2] MDN Web Docs. Array — https://developer.mozilla.org/docs/Web/JavaScript/Reference/Global_Objects/Array

[3] Node.js. File system — <https://nodejs.org/api/fs.html>

[4] Node.js. Readline — <https://nodejs.org/api/readline.html>

[5] Node.js. Test runner — <https://nodejs.org/api/test.html>